

가볼래 포팅 매뉴얼

GitLab 저장소를 클론해 빌드·배포하는 방법, 외부 서비스, DB 덤프, 시연 시나리오를 정리했다. 값은 2026 년 9 월 28 일 저장소와 운영 서버에서 확인했다.

기준 저장소: S15P21E201 / 작성일: 2026 년 9 월 28 일

1. 빌드 및 배포

1) 사용 제품·설정 값·버전

구분	제품·버전	주요 설정 값
운영 서버	Ubuntu 24.04.4 LTS, Docker 29.7.2	컨테이너 네트워크 local-route-personalization_data_net, 재시작 정책 unless-stopped
웹서버	Nginx 1.24.0 (호스트), 프론트 컨테이너 내부 nginx:alpine	443 SSL(Let's Encrypt, Certbot), /api/ → localhost:8080, / → localhost:3000, /jenkins/ → 127.0.0.1:8081, client_max_body_size 5m
WAS	Spring Boot 4.0.8 내장 Tomcat 11.0.25	포트 8080, 업로드 한도 max-file-size=4MB, max-request-size=4608KB
JVM	빌드 eclipse-temurin:17-jdk, 실행 eclipse-temurin:17-jre(운영 OpenJDK 17.0.20)	JAVA_TOOL_OPTIONS=-Xmx1536m(운영), 이미지 기본값 - XX:MaxRAMPercentage=75.0 -Dfile.encoding=UTF-8 -Duser.timezone=Asia/Seoul, 컨테이너 메모리 2g
백엔드 빌드	Java 17, Gradle Wrapper 9.7.1	./gradlew bootJar, JPA ddl-auto=validate, Flyway 사용(스키마 gabolle)
경로 최적화	Python 3.13(OR-Tools 9.14)	백엔드 이미지에 포함, GABOLLE_ROUTE_OPTIMIZER_PYTHON 으로 경로 지정
프론트엔드	Expo ~57.0.22, React Native 0.86.3, 빌드 이미지 node:20-alpine	npm run build:web 으로 정적 파일 생성 후 Nginx 컨테이너 80 → 호스트 3000
모바일 앱	EAS Build	frontend/eas.json, frontend/app.config.js
DB	PostgreSQL 16.4	서비스 DB app_db, 스키마 gabolle, 호스트 포트 미공개(컨테이너 네트워크 안에서만 5432)
CI/CD	Jenkins 2.568.3(jenkins/jenkins:lts, JVM Temurin 21.0.12.1)	백엔드 Job 은 back/dev, 프론트 Job 은 front/dev 체크아웃
IDE	IntelliJ IDEA 2026.2.0.1(백엔드), Visual Studio Code 1.137.0(프론트엔드)	로컬 JDK 17.0.12, Node.js 24.18.0. 저장소에 IDE 버전 고정 파일은 없다

버전은 2026-09-28 운영 서버와 backend/build.gradle, 각 Dockerfile, Jenkinsfile 에서 확인했다. Nginx 경로 설정은 docs/SERVER-SETUP.md 기준이다. IDE 는 작성자 개발 PC 기준이다.

2) 빌드 환경 변수

```
git clone https://lab.ssafy.com/s15-bigdata-dist-sub1/S15P21E201.git
cd S15P21E201/backend
./gradlew bootJar
docker build -t gabolle-backend:local .
cd ../frontend
npm ci
docker build -t gabolle-frontend:local \
  --build-arg EXPO_PUBLIC_API_BASE_URL="https://<도메인>" \
  --build-arg EXPO_PUBLIC_OAUTH_CALLBACK_BASE_URL="https://<도메인>" \
  --build-arg EXPO_PUBLIC_GOOGLE_CLIENT_ID="<공개 ID>" \
  --build-arg EXPO_PUBLIC_NAVER_CLIENT_ID="<공개 ID>" \
  --build-arg EXPO_PUBLIC_KAKAO_CLIENT_ID="<공개 ID>" \
  --build-arg EXPO_PUBLIC_APPLE_CLIENT_ID="<공개 ID>" \
  --build-arg EXPO_PUBLIC_KAKAO_MAP_JS_KEY="<지도 JavaScript 키>" .
```

프론트 변수는 웹 빌드 때 번들에 들어가므로 이미지 빌드 시점에 넘긴다. 백엔드 변수는 컨테이너 실행 시점에 -e 로 넣는다. 운영에서는 Jenkins Credentials 가 값을 보관하고 backend/Jenkinsfile 의 withCredentials 가 주입한다.

변수	용도	주입 시점
EXPO_PUBLIC_API_BASE_URL	백엔드 주소. /api/v1 과 끝 슬래시를 붙이지 않는다	프론트 빌드
EXPO_PUBLIC_OAUTH_CALLBACK_BASE_URL	소셜 로그인 콜백 기준 주소	프론트 빌드
EXPO_PUBLIC_{GOOGLE,NAVER,KAKAO,APPLE}_CLIENT_ID	소셜 로그인 공개 client ID	프론트 빌드
EXPO_PUBLIC_KAKAO_MAP_JS_KEY	웹.앱 지도 JavaScript 키	프론트 빌드
SPRING_PROFILES_ACTIVE	실행 프로필. 운영은 dev	백엔드 실행
GABOLLE_DB_URL, GABOLLE_DB_USERNAME, GABOLLE_DB_PASSWORD, GABOLLE_DB_SCHEMA	PostgreSQL 접속과 Flyway 대상 스키마	백엔드 실행
GABOLLE_JWT_SECRET	액세스 토큰 서명 비밀값(필수)	백엔드 실행
GABOLLE_OAUTH_ALLOWED_REDIRECT_URIS, GABOLLE_CORS_ALLOWED_ORIGINS	허용 콜백 주소와 허용 웹 origin	백엔드 실행
GABOLLE_{GOOGLE,NAVER,KAKAO}_CLIENT_ID, ... _CLIENT_SECRET	소셜 로그인 서버 자격증명	백엔드 실행
GABOLLE_APPLE_CLIENT_ID, _TEAM_ID, _KEY_ID, _PRIVATE_KEY	Apple 로그인 서버 자격증명	백엔드 실행

변수	용도	주입 시점
GABOLLE_MAIL_ENABLED, GABOLLE_MAIL_HOST, _PORT, _USERNAME, _PASSWORD, _FROM	인증 메일 발송(Gmail SMTP)	백엔드 실행
GABOLLE_STORAGE_PROVIDER, GABOLLE_S3_ENDPOINT, _BUCKET, _ACCESS_KEY, _SECRET_KEY, _PUBLIC_BASE_URL	사진 저장소(MinIO)	백엔드 실행
GABOLLE_MENU_SCAN_API_KEY, GABOLLE_ASSISTANT_API_KEY	SSAFY GMS(메뉴판 읽기·여행 이름·번역), 자연어 도우미	백엔드 실행
GABOLLE_KMA_SERVICE_KEY, GABOLLE_TRANSIT_SERVICE_KEY, GABOLLE_EXCHANGE_RATE_AUTH_KEY, GABOLLE_KAKAO_REST_API_KEY	날씨, 버스 도착정보, 환율, 길찾기	백엔드 실행
GABOLLE_INTERNAL_API_TOKEN, GABOLLE_ADMIN_EMAILS	내부 배치 API 토큰, 관리자 계정 목록	백엔드 실행

로컬에서는 backend/.env.example 을 backend/.env 로 복사해 값을 채운 뒤 backend/에서 docker compose up -d --build 로 백엔드와 PostgreSQL 을 함께 띄운다. POSTGRES_PASSWORD 와 GABOLLE_JWT_SECRET 은 필수다.

3) 배포 시 특이사항

1. 운영 Jenkins 는 back/dev, front/dev 를 배포한다. main 에 머지해도 자동 배포되지 않는다.
2. 백엔드 배포는 운영 DB 에 적용된 Flyway 버전과 새 SQL 순서를 먼저 검사한다. 새 이미지를 candidate 로 띄워 /actuator/health 가 정상일 때만 latest 로 올린다. 프론트도 같은 방식이다.
3. Nginx 의 /api/ proxy_pass 끝에 /를 붙이면 /api 접두사가 사라진다. proxy_pass http://localhost:8080;로 둔다.
4. Nginx 기본 업로드 한도가 1MB 라 사진 업로드를 위해 client_max_body_size 5m 을 둔다. 설정 파일은 /etc/nginx/conf.d/upload-size.conf 다.
5. EXPO_PUBLIC_*가 빠져도 프론트 빌드는 경고만 내고 끝난다. 배포 뒤 로그인·지도·API 연결을 화면에서 확인한다.
6. 운영 DB 명은 app_db, 로컬 Compose 기본 DB 명은 gabolle 이다.

4) DB 접속 정보와 프로퍼티 파일 목록

파일	정의 내용
backend/src/main/resources/application.properties	공통 설정, 업로드 한도
backend/src/main/resources/application-dev.properties	데이터소스(GABOLLE_DB_*), Flyway, 인증, 외부 서비스 설정

파일	정의 내용
backend/.env.example, backend/docker-compose.yml	로컬 DB 이름·계정·비밀번호 변수와 컨테이너 전달 목록
backend/Jenkinsfile, frontend/Jenkinsfile	운영 Jenkins Credentials ID 와 컨테이너 전달값
infra/personalization/.env.example, compose.yaml, postgres/init/01-init-databases.sh	운영 PostgreSQL 관리자와 app_db·airflow_db·mlflow_db 계정 초기화
backend/src/main/resources/db/migration/	Flyway 스키마 이력(ERD 의 근거)
frontend/.env.example, frontend/app.config.js, frontend/eas.json	웹·앱 빌드 변수

ERD 는 [GABOLLE ERD \(FigJam\)](#)에 있다. 실제 값은 저장소에 없고 운영 Jenkins Credentials 에만 있다.

운영 DB 는 호스트 포트가 열려 있지 않아 사무국 지급 PEM 으로 서버에 SSH 접속한 뒤 컨테이너 안에서 접속한다. 2026-09-28 에 아래 명령으로 접속과 조회를 확인했다.

```
ssh -i <지급받은 키>.pem ubuntu@j15e201.p.ssafy.io
docker ps --format '{{.Names}} {{.Image}}' # local-route-personalization-postgres-1
docker exec -it local-route-personalization-postgres-1 psql -U postgres -d app_db
```

Linux·macOS 에서는 먼저 `chmod 600 <지급받은 키>.pem` 으로 권한을 줄인다. 같은 인스턴스에 `airflow_db`, `mlflow_db` 가 함께 있고 서비스 데이터는 `app_db` 의 `gabolle` 스키마에 있다.

2. 외부 서비스

키 값과 콘솔 계정은 심지 않았다. 발급받은 값은 1 장 2)의 변수에 넣는다.

서비스	가입·등록 방법	사용하는 변수
Google 로그인	Google Cloud Console 에서 OAuth 클라이언트(웹)를 만들고 승인된 리디렉션 URI 에 콜백 주소를 등록한다	EXPO_PUBLIC_GOOGLE_CLIENT_ID, GABOLLE_GOOGLE_CLIENT_ID, GABOLLE_GOOGLE_CLIENT_SECRET
Naver 로그인	NAVER Developers 에서 애플리케이션을 등록하고 네아로 Callback URL 을 입력한다	EXPO_PUBLIC_NAVER_CLIENT_ID, GABOLLE_NAVER_CLIENT_ID, GABOLLE_NAVER_CLIENT_SECRET
Kakao 로그인·지도	Kakao Developers 에서 앱을 만들고 카카오 로그인 Redirect URI, 웹 사이트 도메인을 등록한다. REST API 키는 로그인과 길찾기, JavaScript 키는 지도에 쓴다	EXPO_PUBLIC_KAKAO_CLIENT_ID, EXPO_PUBLIC_KAKAO_MAP_JS_KEY, GABOLLE_KAKAO_CLIENT_ID, GABOLLE_KAKAO_CLIENT_SECRET, GABOLLE_KAKAO_REST_API_KEY

서비스	가입·등록 방법	사용하는 변수
Apple 로그인	Apple Developer 에서 Service ID, Team ID, Key ID 와 .p8 키를 발급하고 Return URL 을 등록한다	EXPO_PUBLIC_APPLE_CLIENT_ID, GABOLLE_APPLE_CLIENT_ID, _TEAM_ID, _KEY_ID, _PRIVATE_KEY
SSAFY GMS	SSAFY 에서 GMS 키를 발급받는다. 메뉴판 읽기, 여행 이름 추천, 번역이 같은 키를 쓴다	GABOLLE_MENU_SCAN_API_KEY
Google Gemini	Google AI Studio 에서 API 키를 발급받는다. 자연어 도우미에 쓴다	GABOLLE_ASSISTANT_API_KEY
공공데이터포털	data.go.kr 가입 후 TourAPI 관광정보, 무장애 여행정보, 부산 버스 위치(BIMS), TAGO 정류소·도착정보를 활용신청한다	DATA_GO_KR_KEY(수집), GABOLLE_TRANSIT_SERVICE_KEY(서버)
기상청 단기예보	기상청 API 허브에서 인증키를 발급받는다. data.go.kr 키와 다르다	GABOLLE_KMA_SERVICE_KEY
한국수출입은행 환율	한국수출입은행 Open API 에서 인증키를 발급받는다	GABOLLE_EXCHANGE_RATE_AUTH_KEY
Mapillary	Mapillary 개발자 페이지에서 앱을 등록하고 토큰을 발급받는다. 거리 이미지 수집에 쓴다	MAPILLARY_TOKEN
Gmail SMTP	발송 계정에 2 단계 인증을 켜고 앱 비밀번호를 만든다	GABOLLE_MAIL_*
Expo EAS	Expo 계정으로 프로젝트에 접근해 앱을 빌드한다	frontend/eas.json 프로필
MinIO	별도 가입 없이 운영 서버 Docker 로 띄운 S3 호환 저장소다	GABOLLE_S3_*

소셜 로그인 콜백 주소는 아래와 같다. 도메인을 바꾸면 제공자 콘솔, EXPO_PUBLIC_OAUTH_CALLBACK_BASE_URL, GABOLLE_OAUTH_ALLOWED_REDIRECT_URI 를 같은 값으로 맞춘다. Google 은 대소문자와 끝 슬래시까지 일치해야 한다.

```

https://j15e201.p.ssafy.io/oauth/google/callback
https://j15e201.p.ssafy.io/oauth/naver/callback
https://j15e201.p.ssafy.io/oauth/kakao/callback
https://j15e201.p.ssafy.io/oauth/apple/callback
https://j15e201.p.ssafy.io/api/v1/auth/oauth/apple/form-post

```

3. DB 덤프 최신본

운영 app_db 를 2026-09-28 09:30 KST 에 pg_dump -Fc 로 추출했다. 원본에는 팀원과 테스터의 이메일, 비밀번호 해시, 아직 유효한 초대 링크 토큰, 기기 GPS 좌표가 들어 있어서 개인정보를 가린 사본을 제출한다.

파일	내용	Git
exec/app_db_masked_20260928.dump	개인정보 마스킹 사본. 53,920,512 바이트, SHA-256 ab2de497cbcd59cb81bb40a2d13aaab7576d9cedce37c537be1f4ed994a9cf5d	포함
exec/app_db_exec_20260928T003005Z.dump	운영 원본. 54,406,023 바이트, SHA-256 aa53c0a8c95ce349c51c9f5a550deaf6dd780f2ea43db4d0cfb1914198679032	제외

마스킹 내용

대상	처리
로컬 계정 이메일·비밀번호	user<N>@example.com, 공통 비밀번호 gabolle-demo-2026 으로 변경
소셜 연동 이메일·제공자 ID	social<N>@example.com, masked-<제공자>-<N>으로 변경
표시 이름	여행자<N>으로 변경
여행 초대·공유·스토리 초대 토큰, 푸시 토큰	새 난수로 변경
여행 출발지 좌표	소수 셋째 자리(약 100m)로 반올림
세션·리프레시 토큰, 일회용 토큰, OAuth 진행 기록	삭제(auth_session, auth_refresh_token, auth_one_time_token, oauth_challenge, oauth_signup_ticket)

장소, 추천, 일정, 스토리 등 나머지 데이터는 그대로다. 로컬 PostgreSQL 16.4 에서 원본을 복원해 마스킹하고, 결과를 새 DB 에 다시 복원해 확인했다. 복원 오류는 없었고 88 개 테이블의 행 수는 삭제한 5 개 테이블을 빼면 원본과 같다. 원본 이메일·토큰·표시 이름을 사본 전체에서 대조한 결과 남은 값은 없다.

복원 방법

```
createdb -U postgres app_db
pg_restore -U postgres -d app_db --no-owner --no-acl exec/app_db_masked_20260928.dump
```

PostgreSQL 16 이상에서 복원한다. 여행 데이터가 가장 많은 활성 계정은 user2@example.com 이고 비밀번호는 gabolle-demo-2026 이다. 저장된 bcrypt 해시가 이 비밀번호와 맞는 것까지 확인했고, 복원 DB 에 백엔드를 붙여 실제로 로그인해 보지는 않았다. 소셜 로그인 계정은 제공자 ID 를 바꿨기 때문에 복원 환경에서는 로그인되지 않는다.

운영 DB 에서 새로 추출하는 방법

1 장 4)의 방법으로 서버에 접속한 뒤 실행한다. 원본은 개인정보가 있으므로 Git 과 메신저에 올리지 않는다.

```
umask 077
dump_file="$HOME/app_db_$(date -u +%Y%m%dT%H%M%S).dump"
docker exec local-route-personalization-postgres-1 pg_dump -U postgres -Fc app_db > "$dump_file"
sha256sum "$dump_file"
```

로컬로 가져올 때는 `scp -i <지금받은 키>.pem ubuntu@j15e201.p.ssafy.io:<서버 덤프 경로> .`를 쓴다.

4. 시연 시나리오

시연은 앱으로 여행 만들기 기능을 1 회 실시한다. 여행 초안을 생성하고 수정한 뒤 내 여행으로 확정하는 과정까지 보여 주고, 내 여행 기능 발표로 넘어간다. 아래는 같은 흐름을 웹(<https://j15e201.p.ssafy.io>) 기준으로 적은 것이다. 앱은 같은 화면을 하단 탭 막대로 이동한다.

순서	화면	클릭 위치와 조작	결과
1	/sign-in	우측 상단 「로그인」을 누르고 이메일과 비밀번호를 입력한 뒤 「로그인」	홈으로 이동하고 우측 상단에 이름이 표시된다
2	/	우측 상단 「여행 만들기」(그림 1 의 ①)	여행 조건 화면으로 이동한다. 인원은 홈 가운데 검색 줄의 「인원」에서 정하며 기본값은 성인 2 명이다
3	/plan	「어디에서 출발하세요?」 카드(그림 2 의 ①)를 눌러 출발지를 고른다. 「언제 가세요?」 달력(②)에서 출발일, 귀환일 순으로 누르거나 「1 박 2 일」 같은 칩을 누른다	왼쪽 「이번 여행」 카드에 출발지와 날짜가 표시된다
4	/plan	왼쪽 목록(③)의 순서대로 여행 범위, 총예산, 하루 여행 시간·이동수단, 여행 카테고리, 여행 기분, 이번 여행 이동 보조·짐, 꼭 가고 싶은 장소를 오른쪽 카드에서 고르고 하단 「다음」을 누른다. 선택 질문은 카드 우측 상단 「건너뛰기」로 넘길 수 있다	마지막 질문에서 버튼이 「이 조건으로 일정 만들기」로 바뀐다
5	/plan	「이 조건으로 일정 만들기」	생성 화면으로 이동하며 요청마다 새 jobId 가 붙는다
6	/plan/generating?jobId=...	상단 띠(그림 3 의 ①)의 네 단계가 모두 「완료」가 될 때까지 기다린 뒤 가운데 승차권(②)을 누른다. 뒤집힌 면의 「일정 보기 →」를 누른다	추천 코스 화면으로 이동한다

순서	화면	클릭 위치와 조작	결과
7	/trips/{id}/recommendations	왼쪽 코스 카드의 「이 코스 선택」을 누르고, 바뀐 버튼 「이 코스로 일정 만들기 →」를 누른다. 이름 붙이기 창에서 추천 이름의 「이 이름으로 →」 또는 「저장」	선택한 코스가 확정되어 내 여행에 저장되고 일정 화면이 열린다
8	/trips/{id}/itinerary	상단 「코스 A 확정」 표시(그림 4의 ①)를 보여 준다. 우측 상단 「...」(②) → 「일정 편집」(③)을 누르고 「순서 변경」으로 장소 순서를 바꾼 뒤 「저장」	바뀐 순서와 이동 시간이 일정에 반영된다
9	/trips	상단 가운데 「내 여행」(그림 5의 ①)을 누르고 방금 만든 여행 카드(②)를 연다	여기서 내 여행 기능 발표로 넘어간다

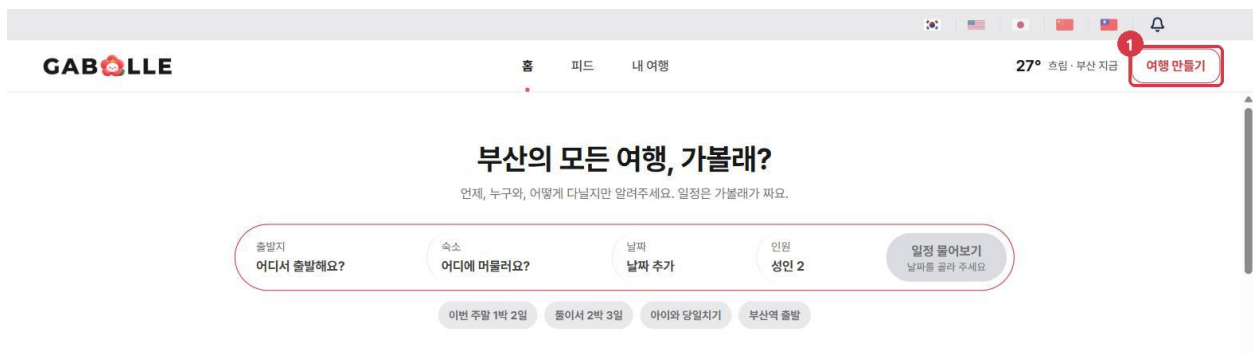


그림 1. 홈 — 우측 상단 「여행 만들기」

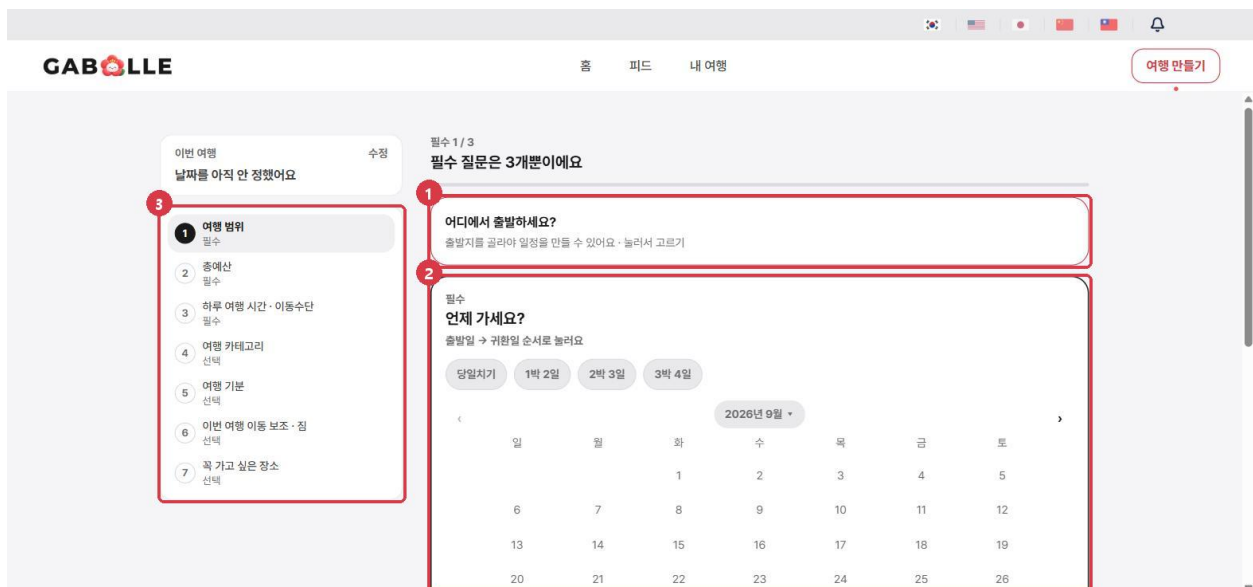


그림 2. 여행 조건 — ① 출발지, ② 날짜, ③ 질문 목록

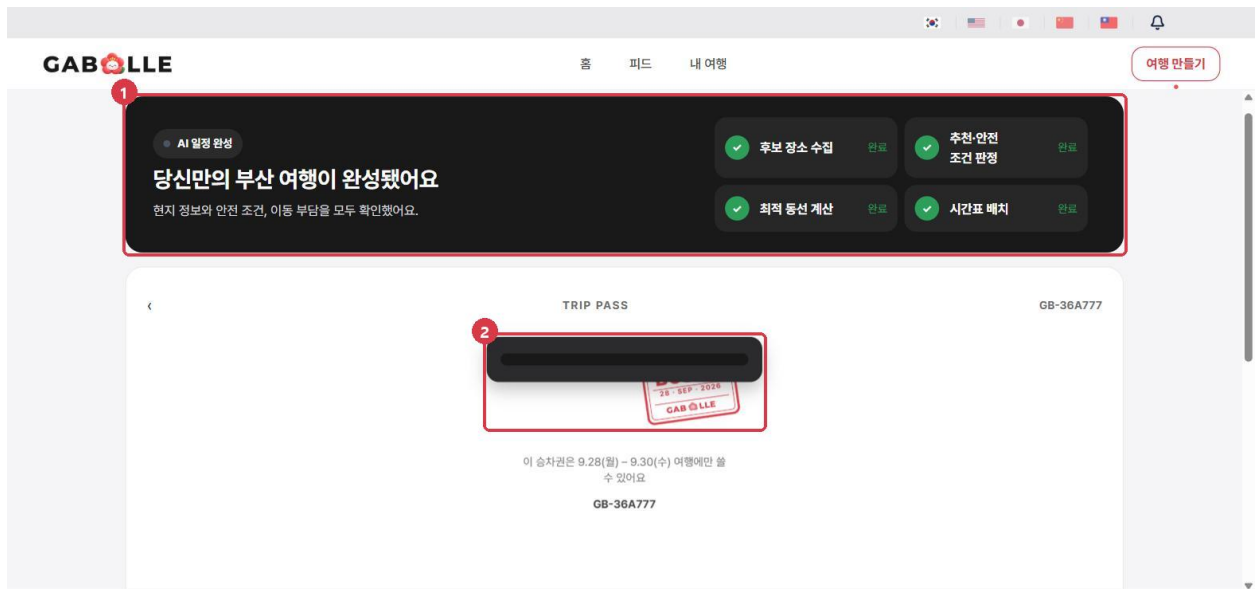


그림 3. 일정 생성 — ① 진행 단계, ② 승차권

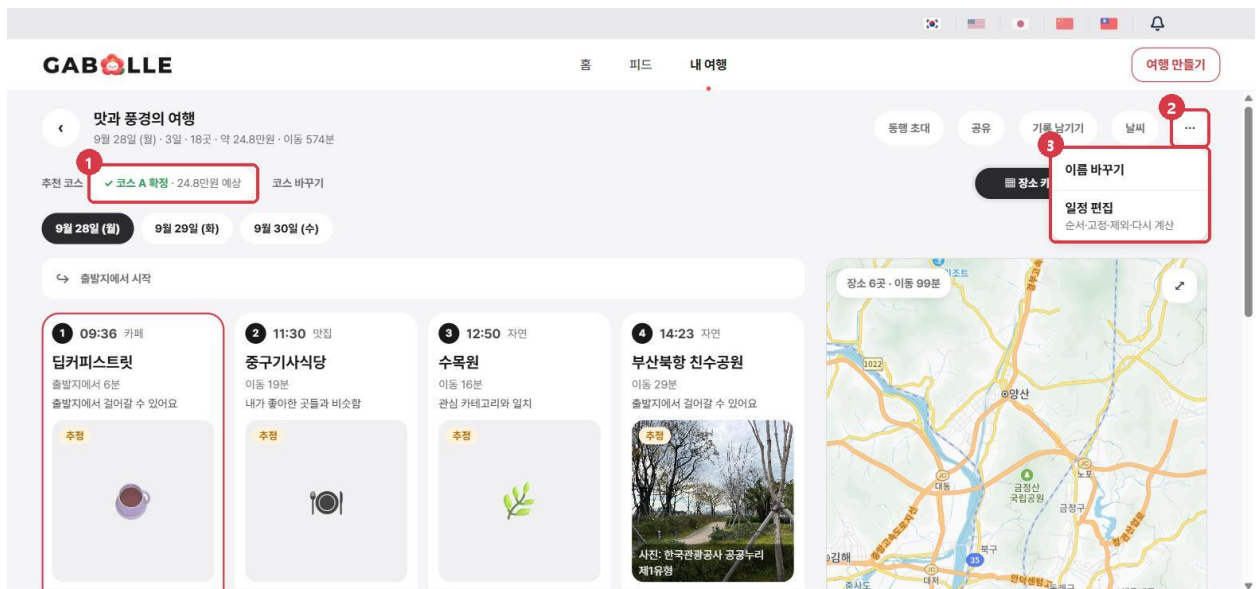


그림 4. 일정 — ① 확정된 코스, ② 더보기, ③ 일정 편집

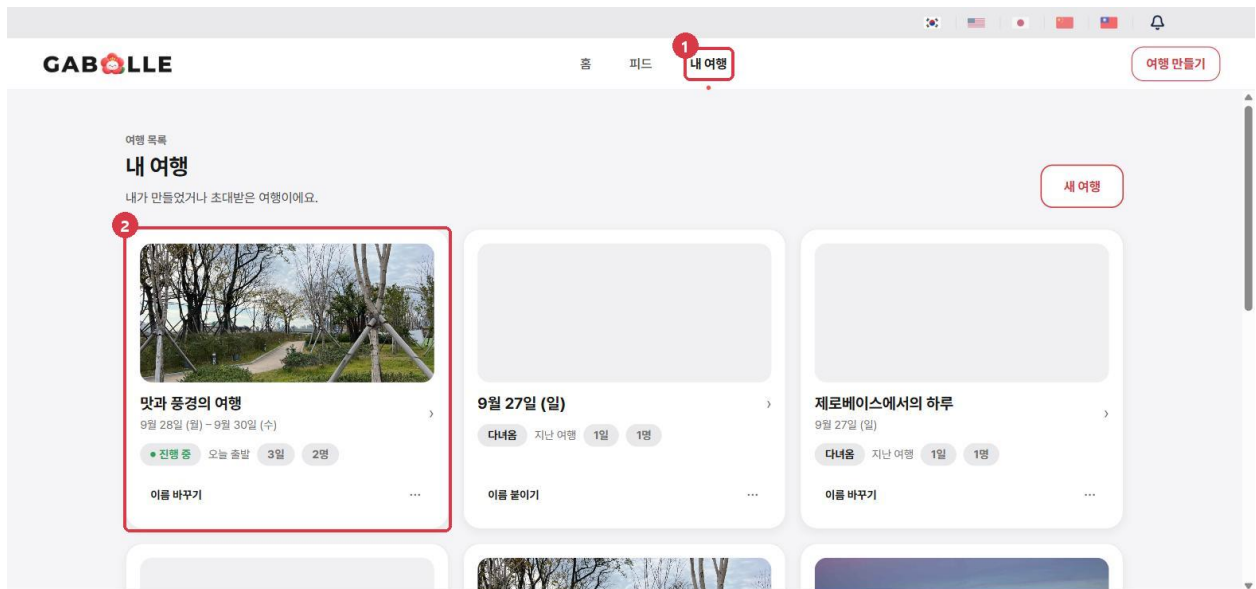


그림 5. 내 여행 — ① 상단 메뉴, ② 여행 카드

생성이 오래 걸리면 진행 단계를 설명하며 기다린다. 실패 안내가 뜨면 「조건 다시 확인하기」로 돌아가 입력과 네트워크를 확인한다. 생성은 1 회만 하고, 다시 시도했다면 그 사실을 말한다.